IPC

IPC in Shared-Memory Systems Interprocess communication using shared memory requires communicating processes to establish a region of shared memory. Typically, a shared-memory region resides in the address space of the process creating the shared-memory segment. Other processes that wish to communicate using this shared-memory segment must attach it to their address space. Recall that, normally, the operating system tries to prevent one process from accessing another process's memory. Shared memory requires that two or more processes agree to remove this restriction. They can then exchange information by reading and writing data in the shared areas. The form of the data and the location are determined by these processes and are not under the operating system's control. The processes are also responsible for ensuring that they are not writing to the same location simultaneously

To illustrate the concept of cooperating processes, let's consider the producer–consumer problem, which is a common paradigm for cooperating processes. A producer process produces information that is consumed by a consumer process.

One solution to the producer–consumer problem uses shared memory. To allow producer and consumer processes to run concurrently, we must have available a buffer of items that can be filled by the producer and emptied by the consumer. This buffer will reside in a region of memory that is shared by the producer and consumer processes. A producer can produce one item while the consumer is consuming another item. The producer and consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been produced.

Two types of buffers can be used. The unbounded buffer places no practical limit on the size of the buffer. The consumer may have to wait for new items, but the producer can always produce new items. The bounded buffer assumes a fixed buffer size. In this case, the consumer must wait if the buffer is empty, and the producer must wait if the buffer is full. Let's look more closely at how the bounded buffer illustrates interprocess communication using shared memory.

Producer–Consumer using Shared Memory (Bounded Buffer)

What is this code trying to show?

This code demonstrates **Interprocess Communication (IPC)** using **shared memory**, where:

- **Producer process** → puts items into a buffer
- **Consumer process** → takes items out of the buffer
- Both processes **share the same memory region**

Shared Data Structures (IMPORTANT)

```
#define BUFFER_SIZE 10

typedef struct {
  ...
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

What these mean:

Variable	Purpose
buffer[BUFFER_SIZE]	Shared circular buffer
in	Index where the **producer inserts** the next item
out	Index where the **consumer removes** the next item

Circular Buffer Concept

The buffer is treated as a **circular array**, not a normal array.

Variable	Purpose

Why only BUFFER_SIZE – 1 items?

Even though BUFFER_SIZE = 10, the buffer can hold **only 9 items**.

Why?
Because:

- in == out is used to detect **empty**
- So one slot must remain empty to **differentiate full vs empty**

This avoids ambiguity.

3.6 IPC in Message-Passing Systems

- **Basic Idea**

In **message-passing systems**, **processes do NOT share memory**.
Instead, they **communicate by exchanging messages** using **operating-system support**.

Comparison with Shared Memory (Section 3.5)

Shared Memory IPC	Message Passing IPC
Processes share a memory region	No shared memory
Programmer manages synchronization	OS manages communication
Faster (no kernel involvement after setup)	Slower (kernel involved)
Risk of race conditions	Safer and easier to implement
Requires explicit access code	Uses send/receive primitives

Message passing provides a mechanism to allow processes to communicate and to synchronize their actions without sharing the same address space. It is particularly useful in a distributed environment, where the communicating processes may reside on different computers connected by a network. For example, an Internet chat program could be designed so that chat participants communicate with one

another by exchanging messages. A message-passing facility provides at least two operations: send(message) and receive(message)

```
#include <stdio.h>


#define MAX_PROC 5   // number of processes


// One mailbox per process
int mailbox[MAX_PROC];
int hasMessage[MAX_PROC] = {0}; // 0 = empty, 1 = full


/* send(P, message)
  P = destination process ID
*/
void send(int P, int message) {


   // BASE CASE: mailbox is empty
   if (hasMessage[P] == 0) {
      mailbox[P] = message;
      hasMessage[P] = 1;
      printf("Message %d sent to process %d\n", message, P);
      return;
   }


   // RECURSIVE CASE: mailbox full → wait
   send(P, message);
}
```

```
/* receive(Q)
  Q = receiving process ID
*/
int receive(int Q) {

  // BASE CASE: message available
  if (hasMessage[Q] == 1) {
    int msg = mailbox[Q];
    hasMessage[Q] = 0;
    printf("Process %d received message %d\n", Q, msg);
    return msg;
  }

  // RECURSIVE CASE: mailbox empty → wait
  return receive(Q);
}

int main() {

  int receivedMessage;

  // Process 0 sends a message to Process 1
  send(1, 100);

  // Process 1 receives the message
```

```
    receivedMessage = receive(1);


    return 0;
}
```

3.6.1 Naming

Processes that want to communicate must have a way to refer to each other. They can use either direct or indirect communication. Under direct communication, each process that wants to communicate must explicitly name the recipient or sender of the communication. In this scheme, the send() and receive() primitives are defined as:

• send(P, message)—Send a message to process P.

• receive(Q, message)—Receive a message from process Q.

A communication link in this scheme has the following properties:

• A link is established automatically between every pair of processes that

want to communicate. The processes need to know only each other's

identity to communicate.

3.6 IPC in Message-Passing Systems 129

• A link is associated with exactly two processes.

• Between each pair of processes, there exists exactly one link.

This scheme exhibits symmetry in addressing; that is, both the sender process and the receiver process must name the other to communicate. A variant

of this scheme employs asymmetry in addressing. Here, only the sender names

the recipient; the recipient is not required to name the sender. In this scheme,

the send() and receive() primitives are defined as follows:

• send(P, message)—Send a message to process P.

• receive(id, message)—Receive a message from any process. The variable id is set to the name of the process with which communication has taken place.